

UNIT 3:

CPU Scheduling: Scheduling Concepts, Performance Criteria, Process States, Process Transition Diagram, Schedulers, Process Control Block (PCB), Process address space, Process identification information, Threads and their management, Scheduling Algorithms, Multiprocessor Scheduling. Deadlock: System model, Deadlock characterization, Prevention, Avoidance and detection, Recovery from deadlock.

PART A: CPU SCHEDULING

1 Scheduling Concepts

1.1 What is CPU Scheduling?

CPU Scheduling is the process of selecting one process from the ready queue and allocating CPU to it.

Since only one process can run on a single CPU at a time, OS must decide:

👉 Which process runs next?

1.2 Why Scheduling is Needed?

- Many processes are in ready state
- Only one CPU available
- Need efficient resource usage

Example:

You open:

- Chrome
- VS Code
- Spotify

CPU cannot run all at once (single core).

OS decides order using scheduling.

2 Performance Criteria (Very Important for Exams)

Criteria	Meaning	Goal
----------	---------	------

CPU Utilization	% time CPU busy	Maximize
Throughput	Processes completed per unit time	Maximize
Turnaround Time	Completion time – Arrival time	Minimize
Waiting Time	Time in ready queue	Minimize
Response Time	Time to first response	Minimize

Example (ML training server):

Low response time important for APIs.

3 Process States

(Already studied in Unit 2 but important here also)

States:

- New
- Ready
- Running
- Waiting
- Terminated

4 Process Transition Diagram

a) New → Ready:

When a new process is created, the Operating System (OS) prepares it for execution. After preparation, the OS places the process in the Ready Queue, where it waits for CPU allocation.

b) Ready → Running:

The Operating System selects one process from the Ready Queue and assigns the CPU to it. The selected process then moves to the Running state and starts execution.

c) Running → Terminated:

When a process completes its execution, the Operating System removes it from the system and it enters the Terminated state.

Sometimes the OS may terminate a process due to reasons such as time limit exceeded, memory unavailability, access violation, protection error, or I/O failure.

d) Running → Ready:

If the time slice of the CPU expires or the processor receives an interrupt, the Operating System stops the currently running process and moves it back to the Ready state.

e) Running → Waiting:

A process moves to the Waiting (Blocked) state when it needs to wait for an event to occur, such as completion of an I/O operation or the availability of a required resource.

f) Waiting → Ready:

When the event for which the process was waiting is completed (for example, an I/O operation finishes), the process moves from the Waiting state back to the Ready state to wait again for CPU allocation.

1. Arrival Time (AT)

Arrival Time is the time at which a process enters the ready queue and becomes ready to use the CPU.

2. Burst Time (BT)

Burst Time is the total amount of CPU time required by a process to complete its execution.

3. Starting Time (ST)

Starting Time is the time at which a process starts executing on the CPU for the first time.

4. Completion Time (CT)

Completion Time is the time at which a process finishes its execution completely.

5. Turnaround Time (TAT)

Turnaround Time is the total time taken by a process from arrival to completion.

Formula:

$$TAT = \text{Completion Time} - \text{Arrival Time}$$

6. Waiting Time (WT)

Waiting Time is the total time a process spends waiting in the ready queue before getting CPU execution.

Formula:

$$WT = \text{Turnaround Time} - \text{Burst Time}$$

7. Average Waiting Time

Average Waiting Time is the average of waiting times of all processes.

Formula:

$$\text{Average WT} = (\text{Sum of Waiting Times}) / \text{Number of Processes}$$

8. Average Turnaround Time

Average Turnaround Time is the average of turnaround times of all processes.

Formula:

$$\text{Average TAT} = (\text{Sum of Turnaround Times}) / \text{Number of Processes}$$

Exam Tip:

Always draw neat diagram.

5 Types of Schedulers

5.1 Long Term Scheduler

- Selects processes from job pool
- Controls degree of multiprogramming

5.2 Short Term Scheduler

- Selects from ready queue
- Runs very frequently

5.3 Medium Term Scheduler

- Swaps processes in/out of memory

6 Process Control Block (PCB)

PCB contains:

- PID
- State
- CPU registers
- Memory info
- Scheduling info
- I/O status

Without PCB → OS cannot manage process.

7 Process Address Space

Each process has its own memory space:

- Code section
- Data section
- Heap
- Stack

Example:

Two Python programs cannot directly access each other's memory.

8 Process Identification Information

Each process has:

- PID (Process ID)
- Parent PID
- User ID
- Group ID

In Linux:

```
ps -ef shows process info.
```

Threads and Their Management

9.1 What is a Thread?

Thread is a lightweight process.

Process → Heavy

Thread → Light

Threads share:

- Same memory
- Same resources

But each has:

- Own stack
- Own registers

Example:

Browser:

Each tab may run as thread.

9.2 Types of Threads

1. User-level threads
2. Kernel-level threads

Scheduling Algorithms (Very Important)

10.1 FCFS (First Come First Serve)

- Non-preemptive
- Simple
- Can cause long waiting

Problem:

Convoy effect

10.2 SJF (Shortest Job First)

- Smallest burst time first
- Optimal waiting time
- Hard to predict burst time

10.3 Priority Scheduling

- Highest priority first
- Can cause starvation

Solution:

Aging

10.4 Round Robin (RR)

- Time quantum used
- Preemptive
- Good for time sharing systems

If time quantum too large → behaves like FCFS

Too small → too many context switches

1 **1** Multiprocessor Scheduling

Used in multi-core systems.

Two types:

1. Asymmetric multiprocessing
 - a. One master CPU controls others
2. Symmetric multiprocessing (SMP)
 - a. Each processor self-schedules

Important concept:

Load balancing

Example:

Training ML model on multi-core CPU.

PART B: DEADLOCK

1 System Model

System has:

- Processes
- Resources

Resources types:

- Single instance (Printer)
- Multiple instance (Memory blocks)

2 Deadlock Definition

Deadlock is a situation where processes are waiting forever for resources held by each other.

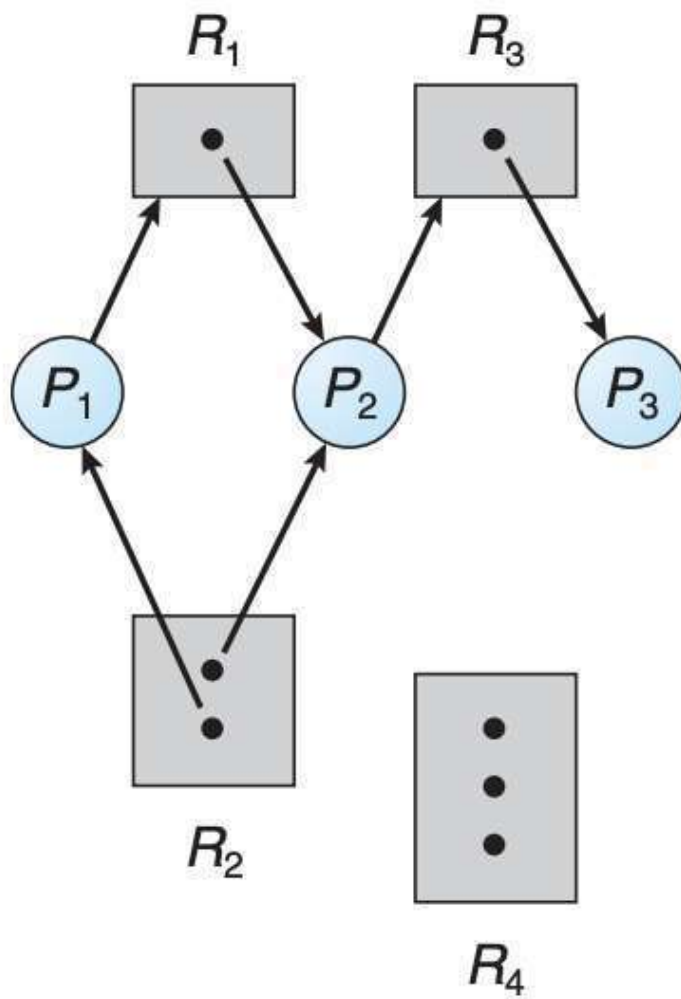
3 Deadlock Characterization

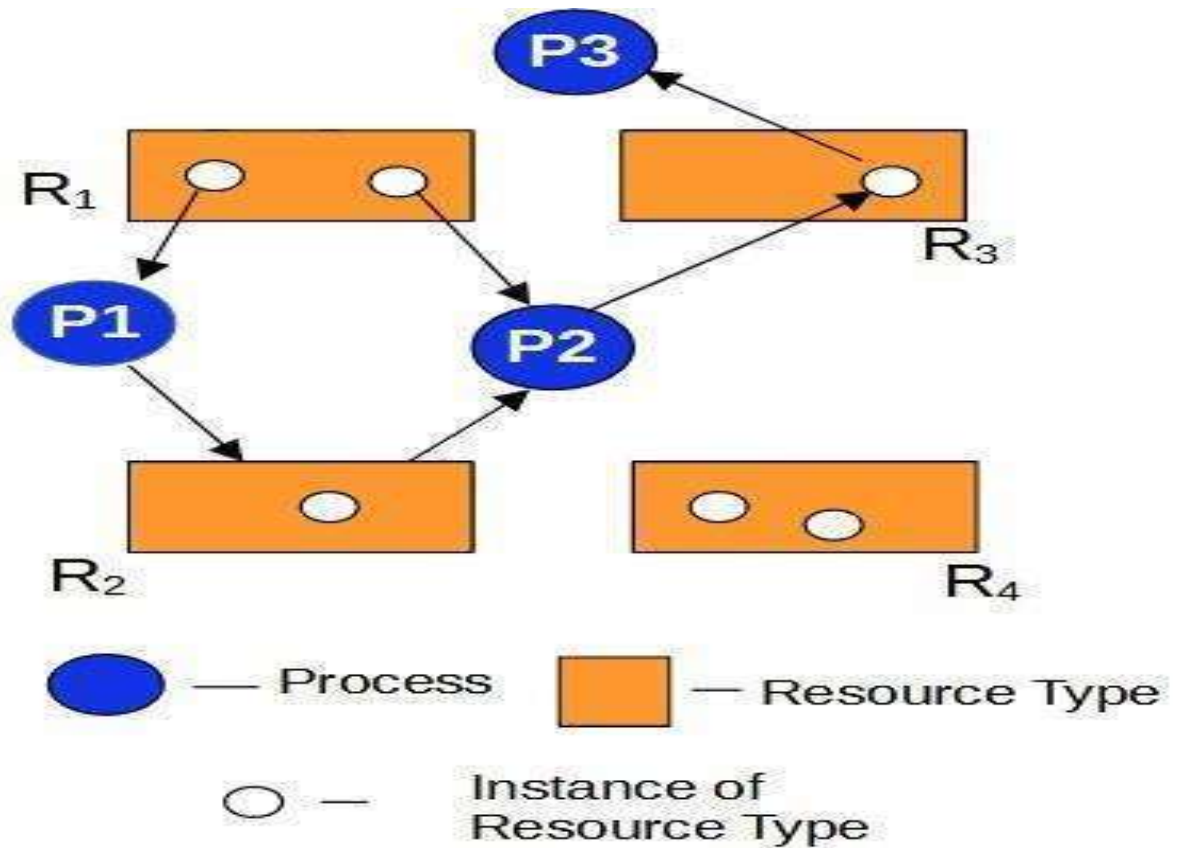
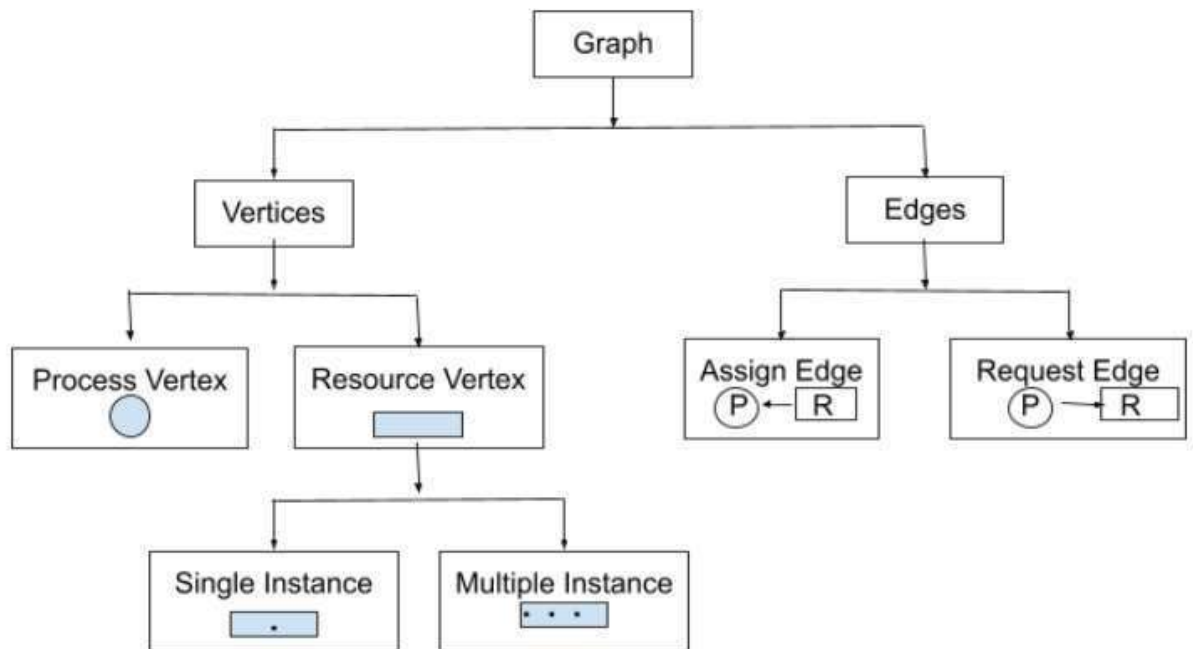
3.1 Necessary Conditions (All 4 Required)

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

If one condition removed → No deadlock.

3.2 Resource Allocation Graph





If cycle present:

- Single instance → Deadlock
- Multiple instance → Possible deadlock

4 Deadlock Prevention

Remove one of 4 conditions.

Example:

Remove Hold and Wait:

Process must request all resources at once.

Problem:

Low resource utilization.

5 Deadlock Avoidance

System checks before allocating resource.

Most famous algorithm:

Banker's Algorithm

Concept:

System only allows safe state.

Safe state = sequence exists to finish all processes.

6 Deadlock Detection

Allow deadlock to occur.

Then detect using:

- Resource allocation graph
- Detection algorithm

7 Recovery from Deadlock

Methods:

1. Process Termination
 - a. Kill one or more processes
2. Resource Preemption
 - a. Take resource back

Problem:

Data loss possible.

Important Difference Table

Prevention	Avoidance	Detection
------------	-----------	-----------

Prevent condition	Check safe state	Detect after occurrence
Conservative	Moderate	Liberal
Low utilization	Better utilization	Highest utilization

1 Preemptive vs Non-Preemptive Scheduling

Basis	Preemptive	Non-Preemptive
CPU Control	CPU can be taken away	CPU not taken until process finishes
Response Time	Better	Poor
Overhead	More (context switching)	Less
Suitable For	Time-sharing systems	Batch systems
Example	Round Robin	FCFS

2 FCFS vs SJF

Basis	FCFS	SJF
Meaning	First come first serve	Shortest job first
Type	Non-preemptive	Can be preemptive or non-preemptive
Waiting Time	Can be high	Minimum average waiting time
Problem	Convoy effect	Need burst time prediction
Starvation	No	Possible (long jobs wait)

3 SJF vs Round Robin

Basis	SJF	Round Robin
Type	Burst time based	Time quantum based
Preemption	Optional	Always preemptive
Response Time	Not good for interactive	Good
Starvation	Possible	No
Best For	Batch processing	Time sharing systems

4 Priority Scheduling vs Round Robin

Basis	Priority	Round Robin
Selection	Highest priority first	Circular order
Starvation	Possible	No
Fairness	Not fair	Fair
Complexity	Medium	Simple
Example	OS kernel tasks	Multi-user systems

5 Long Term vs Short Term Scheduler

Basis	Long Term	Short Term
Also Called	Job Scheduler	CPU Scheduler
Frequency	Runs less often	Runs very frequently
Function	Selects processes from job pool	Selects from ready queue
Controls	Degree of multiprogramming	CPU allocation
Speed	Slower	Very fast

6 Process vs Thread

Basis	Process	Thread
Memory	Separate memory	Shared memory
Creation Cost	High	Low
Communication	IPC required	Direct
Crash Effect	One process crash doesn't affect others	Thread crash may crash whole process
Example	Chrome & VS Code	Multiple tabs in browser

7 Single Processor vs Multiprocessor Scheduling

Basis	Single Processor	Multiprocessor
CPUs	One	Multiple
Execution	One process at a time	Multiple processes simultaneously
Load Balancing	Not required	Required
Performance	Limited	Higher

Example	Basic PC	Multi-core server
---------	----------	-------------------

8 Symmetric vs Asymmetric Multiprocessing

Basis	Symmetric (SMP)	Asymmetric
Control	All CPUs equal	One master CPU
Scheduling	Each CPU self-schedules	Master schedules
Fault Tolerance	High	Lower
Complexity	Higher	Simpler
Example	Modern multi-core systems	Old systems

9 Deadlock vs Starvation

Basis	Deadlock	Starvation
Meaning	Circular waiting forever	Process waits indefinitely
Cause	Resource cycle	Low priority
Recovery	External intervention needed	Priority adjustment
Example	Dining philosophers	Low priority process never runs

10 Deadlock Prevention vs Avoidance vs Detection

Basis	Prevention	Avoidance	Detection
Strategy	Remove one deadlock condition	Check safe state before allocation	Allow then detect
Resource Use	Low utilization	Moderate	High utilization
Complexity	Simple	Complex (Banker's)	Moderate
Example	No hold and wait	Banker's Algorithm	Detection algorithm

1 1 Turnaround Time vs Waiting Time vs Response Time

Basis	Turnaround Time	Waiting Time	Response Time
-------	-----------------	--------------	---------------

Meaning	Completion – Arrival	Time in ready queue	First response – Arrival
Includes Execution?	Yes	No	No
Important For	Batch systems	Efficiency	Interactive systems